

# Instant-Attorney Codebase Refactoring Report

**Date:** June 19, 2026  
**Scope:** `/workspace` monorepo — primary focus on `artifacts/instant-attorney`  
**Type:** Architecture review and refactoring recommendations (no code changes applied)

## Table of Contents

- 1. [Executive Summary](#)
- 2. [Architecture Summary](#)
- 3. [Data Flow](#)
- 4. [Problem Areas](#)
- 5. [Refactoring Strategies](#)
- 6. [Proposed Improved Code \(Illustrative\)](#)
- 7. [Prioritized Roadmap](#)
- 8. [Appendix: Key File Reference](#)

## 1. Executive Summary

This workspace is a **Replit-hosted pnpm monorepo** whose production product is **Instant Attorney** — a Next.js 15 legal-tech application for Crawford Law PLLC. The app guides users through three phases:

| Phase | Audience           | Capability                                                          |
|-------|--------------------|---------------------------------------------------------------------|
| I     | Anonymous visitors | Free legal guidance chat (no auth)                                  |
| II    | Subscribed clients | Privileged intake, Living File, document wizards, attachments       |
| III   | Attorneys          | Document review, critical review, second drafts, consult scheduling |

**Key finding:** The monorepo contains two parallel architectures. The workspace `lib/` packages (Drizzle ORM, OpenAPI spec, generated Zod/React Query clients) and `artifacts/api-server` (Express) are **scaffolding only**. The real product is entirely self-contained in `artifacts/instant-attorney`, which uses **Supabase** directly and is **excluded from the pnpm workspace**.

The application is feature-rich and thoughtfully designed in its AI pipeline (structured output parsing, streaming workarounds, usage tracking). Its main quality risks are **cross-cutting duplication** (~38 API routes each re-implementing auth), **monolithic files** (800+ line modules), **deployment routing gaps**, and **security patterns that depend on application-level checks** rather than database policies.

This report recommends refactoring in layers: fix production routing and security footguns first, extract shared server utilities second, then split large modules — all without changing user-visible behavior.

---

## 2. Architecture Summary

---

### 2.1 Monorepo Layout

```
/workspace/
├── artifacts/
│   ├── instant-attorney/  ★ Production app (Next.js 15, ~136 source files)
│   ├── api-server/       Express 5 stub (health check only)
│   └── mockup-sandbox/    Vite UI sandbox (unrelated to product)
├── lib/
│   ├── db/               Drizzle ORM – empty placeholder schema
│   ├── api-spec/         OpenAPI – /healthz only
│   ├── api-zod/          Generated Zod validators
│   └── api-client-react/  Generated React Query hooks
├── scripts/              Minimal workspace scripts
└── .agents/memory/       Operational knowledge base
```

#### Package manager split:

- Workspace packages: **pnpm** ( `pnpm-workspace.yaml` excludes `instant-attorney` )
- Instant Attorney: **npm** with its own `package-lock.json`

This isolation prevents dependency catalog sharing and creates drift risk between the scaffold stack and the product.

## 2.2 Technology Stack

| Layer      | Technology                                      |
|------------|-------------------------------------------------|
| Framework  | Next.js 15.3.2, React 19, TypeScript 5          |
| Styling    | Tailwind CSS 3                                  |
| Auth & DB  | Supabase Auth + Postgres + Storage              |
| AI         | Anthropic SDK (Sonnet 4.6, Opus 4.8, Haiku 4.5) |
| Payments   | Stripe (checkout, webhooks)                     |
| Email      | Resend (attorney notifications)                 |
| Documents  | docx (generation), mammoth (parsing)            |
| Deployment | Replit dual-service routing                     |

## 2.3 Application Layers

```
Presentation (app/, components/)
- App Router pages, client components, attorney dashboard
```

```
API Layer (app/api/**/route.ts) – 38 route handlers
- Per-route auth, subscription, attorney gates
- AI streaming, document lifecycle, Stripe webhooks
```

```
Domain Logic (lib/) – 30+ modules
- prompts, file-parser, wizard-parsing, document-utils
- mission-control, gov-form-lookup, usage-tracker
```

```
Infrastructure
- supabase/server.ts (user + service clients)
- middleware.ts (page-level session gate)
- instrumentation.ts (crash guard)
```

```
Data (Supabase Postgres + Storage)
- 13 staged SQL migration files (manual apply)
- Row-Level Security (RLS) with known gaps
```

## 2.4 Deployment Topology (Replit)

Two services share the `/api` namespace:

| Service          | Port  | Responsibility                                  |
|------------------|-------|-------------------------------------------------|
| api-server       | 8080  | Owns /api globally; currently only /api/healthz |
| instant-attorney | 21203 | Owns specific paths in artifact.toml            |

**Registered paths** (artifacts/instant-attorney/.replit-artifact/artifact.toml):

```
/, /api/chat, /api/chat-acp, /api/wizard, /api/attachments,
/api/documents, /api/gov-forms, /api/auth, /api/attorney,
/api/subscriptions, /api/agreements, /api/consult, /consult
```

**Missing from routing (likely 404 in production):**

- /api/case-files — archive, merge, answer-gap, restore
- /api/usage — admin usage summary

2.5 Core Domain Entities

| Entity      | Table                                 | Role                                                                 |
|-------------|---------------------------------------|----------------------------------------------------------------------|
| Living File | case_files                            | Central case container: summary, goals, legal strategy, jurisdiction |
| Facts       | fact_items                            | Confirmed facts and information gaps                                 |
| Messages    | intake_messages                       | ACP chat history                                                     |
| Documents   | documents                             | Wizard-generated drafts with attorney review lifecycle               |
| Attachments | attachments                           | Uploaded files with AI analysis                                      |
| Gov Forms   | form_instruments                      | Government form instruments                                          |
| Consults    | consult_requests                      | Attorney scheduling requests                                         |
| Usage       | usage_events ,<br>usage_period_totals | AI and storage cost tracking                                         |

2.6 Document Lifecycle

```
draft → pending_review → approved / changes_requested → delivered
```

- Primary documents: parent\_document\_id IS NULL
- Derived child documents: critical\_review, second\_draft

- 48-hour attorney SLA tracked via `submitted_at`

## 2.7 AI Integration Model

Each route instantiates its own Anthropic client:

```
const anthropic = new Anthropic({
  apiKey: process.env.Claude_Instant_Attorney,
  maxRetries: 4,
});
```

Structured AI output uses marker blocks parsed by `lib/file-parser.ts` and `lib/wizard-parsing.ts`:

- `---LIVING FILE---`, `---LEGAL STRATEGY---`, `---DRAFT READY---`
- `---GOVERNMENT FORMS---`, `---FILE UPDATE---`

Streaming is mandatory for large `max_tokens` values (SDK rejects non-streaming above ~10 minutes). Routes use `maxDuration = 300` (5 minutes).

## 3. Data Flow

### 3.1 Phase I — Free Chat

```
User → /free-chat → POST /api/chat → Claude Sonnet (no auth)
                               → Plain text stream to browser
```

No persistence. Marketing funnel entry point.

### 3.2 Phase II — Subscribed Client Intake

```
User → /onboarding → agreements + Stripe checkout
      → /chat → POST /api/chat-acp
                ├── Auth + subscription check
                ├── Load/create case file
                ├── Load facts, attachments, requested attachments
                ├── Stream Sonnet response (prefix: case file ID)
                └── Post-stream: save message → parse Living File → gov form lookup → title gen
      → /wizard/[type] → POST /api/wizard
                ├── Auth + subscription + case ownership check
                ├── Load case context (4-table Promise.all)
                ├── Stream Sonnet drafter → parse draft
                └── Service-client write to documents table
      → POST /api/documents/[id]/submit → notify attorney (Resend)
```

### 3.3 Phase III — Attorney Workflow

```
Attorney → /attorney → pending review queue (48h SLA)
  → /attorney/review/[id] → POST /api/attorney/documents/[id]/review
    └─ Sonnet critical review → child document
  → POST /api/attorney/documents/[id]/second-draft
    └─ Haiku fitness check → Opus second draft (NDJSON heartbeats)
  → POST /api/attorney/documents/[id]/merge → apply revisions
  → /consult/schedule → consult_requests
```

### 3.4 Authentication Flow

```
Page routes: middleware.ts (Edge) → session cookie check → redirect
API routes:  per-handler auth (Node) → getUser() → subscription/attorney DB checks
Writes:      often via createServiceClient() after app-level ownership verification
```

**Important:** Middleware explicitly does **not** check subscription or attorney status (requires Node runtime DB queries). Each API route re-implements these checks independently.

### 3.5 Data Access Diagram

```
Browser (Pages/Components)
├── middleware.ts (session gate for pages)
└── API Routes (38 handlers)
    ├── lib/ domain modules
    ├── Supabase Auth + Postgres + Storage
    ├── Anthropic API
    ├── Stripe
    └── Resend
```

---

## 4. Problem Areas

---

Problems are grouped by severity and category.

## 4.1 Structural Problems

### S1. Product Isolated from Monorepo Scaffold

| Aspect   | Detail                                                                               |
|----------|--------------------------------------------------------------------------------------|
| Evidence | <code>pnpm-workspace.yaml</code> line 41: <code>artifacts/!(instant-attorney)</code> |
| Impact   | Two dependency trees, no shared type generation, <code>lib/db</code> unused          |
| Risk     | Engineers may invest in Drizzle/OpenAPI stack that the product never consumes        |

### S2. Missing Production Route Registration

| Aspect   | Detail                                                                                                                   |
|----------|--------------------------------------------------------------------------------------------------------------------------|
| Evidence | <code>artifact.toml</code> paths list vs. existing routes under <code>/api/case-files</code> and <code>/api/usage</code> |
| Impact   | Case file archive/merge/restore and admin usage summary may return 404 behind Replit router                              |
| Risk     | Silent feature breakage in production                                                                                    |

### S3. Manual SQL Migration Drift

| Aspect   | Detail                                                                                              |
|----------|-----------------------------------------------------------------------------------------------------|
| Evidence | 13 staged files in <code>supabase/schema*.sql</code> , no automated migration runner                |
| Impact   | Live DB may lack stages 9–11 ( <code>usage_events</code> , <code>form_instruments</code> , RLS fix) |
| Risk     | Silent <code>PGRST205</code> errors; features appear broken without clear cause                     |

## S4. Monolithic Files

| File                          | Lines | Concern                                                 |
|-------------------------------|-------|---------------------------------------------------------|
| app/wizard/[type]/page.tsx    | 880   | Chat UI + checklist + submit + timeout recovery         |
| lib/doc-generator.ts          | 880   | All DOCX templates in one switch                        |
| lib/prompts.ts                | 858   | All system prompts in one file                          |
| components/ClientFileView.tsx | 660   | Client + attorney modes, docs, wizards, mission control |
| lib/wizard-parsing.ts         | 595   | Dense parsing logic (well-tested but hard to navigate)  |
| app/chat/page.tsx             | 569   | Large client component with inline markdown renderer    |

### 4.2 Duplicated Code

#### D1. Auth Boilerplate (~31 API routes)

The same pattern appears in virtually every authenticated route. **Affected files include:**

- app/api/wizard/route.ts (lines 48–70)
- app/api/chat-acp/route.ts (lines 34–56)
- app/api/case-files/[id]/answer-gap/route.ts
- app/api/documents/[id]/fill-info/route.ts
- app/api/attorney/documents/[id]/review/route.ts
- ...and ~26 more

**Drift risk:** A security fix in one route may not propagate to others.

#### D2. Subscription Gate (at least 2 routes)

Identical block in chat-acp/route.ts and wizard/route.ts:

```
const activeStatuses = ["active", "trialing", "bypass"];
if (!sub || !activeStatuses.includes(sub.status)) {
  return NextResponse.json({ error: "Subscription required" }, { status: 403 });
}
```

#### D3. Attorney Gate (~16 routes)

Repeated in all app/api/attorney/\*\* routes and some document routes.

D4. Case File Context Loading (~6 locations)

Same 4-table `Promise.all` in: wizard, chat-acp, review, second-draft, pre-consult, documents/generate, attachment-processor.

D5. SLA Countdown Logic (3 implementations)

| Component                   | Location                                    |
|-----------------------------|---------------------------------------------|
| <code>ReviewClock</code>    | <code>app/attorney/page.tsx</code> (inline) |
| <code>ReviewSlaClock</code> | <code>components/ReviewSlaClock.tsx</code>  |
| <code>SlaCountdown</code>   | <code>app/attorney/SlaCountdown.tsx</code>  |

All compute `deadline = submittedAt + 48 hours` independently with different urgency thresholds.

D6. Chat Markdown Renderer (2 pages)

`renderContent()` duplicated in `app/free-chat/page.tsx` and `app/chat/page.tsx`.

D7. Dashboard Data Loading

`app/dashboard/[id]/page.tsx` and `app/attorney/file/[caseFileId]/page.tsx` perform near-identical queries.

4.3 Performance Bottlenecks

| ID | Issue                                    | Risk                                                                                                   |
|----|------------------------------------------|--------------------------------------------------------------------------------------------------------|
| P1 | Long-running AI routes (up to 300s)      | Proxy timeouts; client timeout at 290s on wizard                                                       |
| P2 | Synchronous post-stream work in chat-acp | Extended connection time after stream ends                                                             |
| P3 | Attachment processing on upload          | Sonnet analysis blocks upload response                                                                 |
| P4 | Unbounded attorney dashboard queries     | 100 docs + 200 case files, no cursor pagination                                                        |
| P5 | No application-level caching             | Every hit queries Supabase directly                                                                    |
| P6 | Client bundle risk                       | <code>ClientFileView</code> imports from <code>document-utils</code> which imports <code>Resend</code> |

## 4.4 Maintainability & Security Risks

| ID | Issue                           | Detail                                                                 |
|----|---------------------------------|------------------------------------------------------------------------|
| M1 | Service client write workaround | RLS blocks owner writes; security depends on app checks                |
| M2 | fact_items RLS gap              | Checks user_id, not case ownership                                     |
| M3 | Hardcoded Supabase credentials  | <code>next.config.ts</code> lines 4–10                                 |
| M4 | ESLint disabled during builds   | <code>ignoreDuringBuilds: true</code>                                  |
| M5 | Legacy schema artifacts         | <code>consults</code> table, retired <code>pre_warmed</code> status    |
| M6 | Non-standard env var names      | <code>Claude_Instant_Attorney</code> vs <code>ANTHROPIC_API_KEY</code> |
| M7 | Fire-and-forget tasks           | <code>instrumentation.ts</code> swallows rejections                    |
| M8 | Test infrastructure gap         | Known broken <code>mission-control.test.ts</code>                      |

## 5. Refactoring Strategies

All strategies preserve existing behavior.

### 5.1 Layer 0 — Operational Fixes

| Action                                                                                     | Impact                    |
|--------------------------------------------------------------------------------------------|---------------------------|
| Add <code>/api/case-files</code> and <code>/api/usage</code> to <code>artifact.toml</code> | Fixes production 404s     |
| Verify all SQL stages applied in production                                                | Prevents silent DB errors |
| Remove hardcoded Supabase fallbacks from <code>next.config.ts</code>                       | Security hygiene          |
| Re-enable ESLint in CI builds                                                              | Quality gate              |

### 5.2 Layer 1 — Extract Shared Server Utilities

Create `lib/api/` with composable guards:

```

lib/api/
├── auth.ts           requireAuth(), BYPASS handling
├── subscription.ts  requireActiveSubscription()
├── attorney.ts      requireAttorney()
├── case-file.ts     loadCaseFileContext(), assertCaseOwnership()
├── anthropic.ts     shared client factory
└── errors.ts        typed JSON error responses

```

**Benefits:** Single source of truth for auth; ~775 lines of duplication removed across routes.

### 5.3 Layer 2 — Split Monolithic Modules

- `prompts.ts` → `lib/prompts/` (per-feature prompt files + `context.ts`)
- `doc-generator.ts` → `lib/doc-generators/` (per wizard type)
- `wizard/[type]/page.tsx` → `components/wizard/` subcomponents + `useWizardSession` hook

### 5.4 Layer 3 — Unify UI Duplication

| Extract                            | From                            | To                                                                      |
|------------------------------------|---------------------------------|-------------------------------------------------------------------------|
| <code>ChatMessageRenderer</code>   | free-chat + chat pages          | <code>components/ChatMessageRenderer.tsx</code>                         |
| <code>SlaCountdown</code> (single) | 3 SLA implementations           | <code>components/SlaCountdown.tsx</code> with <code>variant</code> prop |
| <code>loadCaseFileDashboard</code> | dashboard + attorney file pages | <code>lib/case-file-loader.ts</code>                                    |

### 5.5 Layer 4 — Client/Server Boundary Cleanup

- Move `isValidWizardType` to `lib/types.ts`
- Split `document-utils` into client-safe and `document-utils.server.ts` with `import "server-only"`
- Prevent Resend from entering client bundle graph

### 5.6 Layer 5 — Monorepo Consolidation (Optional)

- Bring instant-attorney into pnpm workspace
- Generate types from Supabase ( `supabase gen types` )
- Retire or clearly mark unused `lib/db` and `api-server` scaffold

## 5.7 Layer 6 — Performance Hardening

- Extend NDJSON heartbeat pattern to wizard and review routes
- Move post-stream Living File parsing to background ( `after()` or `queue` )
- Paginate attorney dashboard
- Queue attachment processing

---

## 6. Proposed Improved Code (Illustrative)

---

These are recommendations only — no code has been changed in the repository.

### 6.1 Shared Auth Helper

```
// lib/api/auth.ts
import { NextResponse } from "next/server";
import { createClient, createServiceClient } from "@lib/supabase/server";
import { BYPASS_USER_ID } from "@lib/types";
import type { SupabaseClient } from "@supabase/supabase-js";

const BYPASS_AUTH = process.env.BYPASS_AUTH === "true";

export type AuthResult =
  | { ok: true; userId: string; db: SupabaseClient }
  | { ok: false; response: NextResponse };

export async function requireAuth(): Promise<AuthResult> {
  const db = BYPASS_AUTH ? createServiceClient() : await createClient();

  if (BYPASS_AUTH) {
    return { ok: true, userId: BYPASS_USER_ID, db };
  }

  const { data: { user }, error } = await db.auth.getUser();
  if (error || !user) {
    return {
      ok: false,
      response: NextResponse.json({ error: "Unauthorized" }, { status: 401 }),
    };
  }

  return { ok: true, userId: user.id, db };
}
```

## 6.2 Composable Route Guards

```
// lib/api/subscription.ts
const ACTIVE_STATUSES = ["active", "trialing", "bypass"] as const;

export async function requireActiveSubscription(db, userId) {
  const { data: sub } = await db.from("subscriptions")
    .select("status").eq("user_id", userId).maybeSingle();
  if (!sub || !ACTIVE_STATUSES.includes(sub.status)) {
    return NextResponse.json({ error: "Subscription required" }, { status: 403 });
  }
  return null;
}
```

## 6.3 Case File Context Loader

```
// lib/api/case-file.ts
export async function loadCaseFileContext(db, caseFileId) {
  const [caseFile, facts, attachments, requestedAttachments] = await Promise.all([
    db.from("case_files").select("*").eq("id", caseFileId).single(),
    db.from("fact_items").select("*").eq("case_file_id", caseFileId),
    db.from("attachments").select("*").eq("case_file_id", caseFileId).eq("status", "ready"),
    db.from("requested_attachments").select("*").eq("case_file_id", caseFileId),
  ]);
  return { caseFile, facts, attachments, requestedAttachments };
}
```

## 6.4 Refactored Route Handler Pattern

**Before:** ~86 lines of auth/context boilerplate per route.

**After:**

```
export async function POST(req: NextRequest) {
  const body = await parseWizardBody(req);
  if (!body.ok) return body.response;

  const auth = await requireAuth();
  if (!auth.ok) return auth.response;

  const subError = await requireActiveSubscription(auth.db, auth.userId);
  if (subError) return subError;

  if (!await assertCaseOwnership(auth.db, body.caseFileId, auth.userId)) {
    return forbidden();
  }

  const ctx = await loadCaseFileContext(auth.db, body.caseFileId);
  // ... business logic only ...
}
```

## 6.5 Unified SLA Component

Single `SlaCountdown` with `variant: "attorney-table" | "client-full" | "client-compact"` and optional `live` prop for ticking seconds.

## 6.6 Server-Only Module Split

```
// lib/document-utils.ts – client-safe exports only
// lib/document-utils.server.ts – import "server-only"; lifecycle + notify
```

# 7. Prioritized Roadmap

### Phase 1 — Production Safety

- Add missing paths to `artifact.toml`
- Audit production Supabase migration stage
- Remove hardcoded credential fallbacks
- Document required env vars

### Phase 2 — Security Hardening

- Audit all routes writing `fact_items` for case ownership checks
- Fix or document RLS policies; reduce service-client reliance
- Centralize subscription and attorney gate logic

### Phase 3 — DRY Extraction

- Create `lib/api/` helpers
- Migrate routes domain-by-domain (start with `case-files`)
- Extract `ChatMessageRenderer` and unified `SlaCountdown`

### Phase 4 — Module Decomposition

- Split `prompts.ts`, `doc-generator.ts`, `wizard/[type]/page.tsx`

### Phase 5 — Performance & Observability

- NDJSON heartbeats on `wizard/review` routes

- Paginate attorney dashboard
- Structured logging for background task failures

**Phase 6 — Monorepo Alignment (optional)**

- Evaluate pnpm workspace inclusion
- Generate Supabase types
- Retire unused scaffold

---

## 8. Appendix: Key File Reference

---

**Entry Points**

| Path                                                  | Purpose             |
|-------------------------------------------------------|---------------------|
| artifacts/instant-attorney/app/page.tsx               | Marketing landing   |
| artifacts/instant-attorney/app/free-chat/page.tsx     | Phase I free chat   |
| artifacts/instant-attorney/app/chat/page.tsx          | Phase II ACP chat   |
| artifacts/instant-attorney/app/wizard/[type]/page.tsx | Document wizard     |
| artifacts/instant-attorney/app/attorney/page.tsx      | Attorney dashboard  |
| artifacts/instant-attorney/middleware.ts              | Page auth gate      |
| artifacts/instant-attorney/instrumentation.ts         | Process crash guard |

## Domain Modules

| Path                   | Responsibility                      |
|------------------------|-------------------------------------|
| lib/types.ts           | Domain types, wizard labels         |
| lib/prompts.ts         | System prompts and context builders |
| lib/file-parser.ts     | Parse AI blocks → DB updates        |
| lib/wizard-parsing.ts  | Drafter response parsing            |
| lib/document-utils.ts  | Document CRUD and submission        |
| lib/doc-generator.ts   | DOCX generation                     |
| lib/mission-control.ts | Dashboard action queue              |
| lib/usage-tracker.ts   | Cost accounting                     |

## Database Migrations (13 stages)

Core tables in `schema.sql` ; documents in stage 2; attachments in stage 4; SLA in stage 5; usage/gov forms in stages 9–10; RLS fixes in stages 11–13.

## What's Working Well (Preserve These)

1. **Structured AI output parsing** with completeness guards
2. **Staged SQL migrations** with verify script
3. **Usage tracking** — non-blocking, per-model pricing
4. **NDJSON heartbeats** on second-draft route
5. **instrumentation.ts crash guard** for uptime
6. **Explicit IDOR documentation** in wizard route
7. **Unit tests on parsing** modules

*End of report. No source code was modified during this analysis.*